INTAB 최종 개발 결과 보고서

INTAB 인터폰 상태감지 모듈 — 최종 개발 결과 보고서

항목	내용
문서 일자	2026-10-19 (최종)
대상	COMMAX 4선식 아날로그 인터폰 (엘리베이터 비상통화장치), 라인: +12V/24V, Tx, Rx, GND
목적	라인 상태(대기 / 호출 / 통화 / 전원 이상 / 통신 이상)를 감지하여 USB-CDC로 호스트(Windows, Android)에 전달
현재 상태	펌웨어(프론트엔드, 메인) v1.1, Windows 호스트 프로그램 동작 확인 완료. 2시간 20분 연속 시험에서 USB 끊김 없음. Android 앱은 별도 엔지니어가 개발 예정
관련 문서	인터폰_상태감지_개발사양서.md (v2.2), 통신_프로토콜_정리.md (v1.5), 프론트엔드_MCU_개발사양서.md, 태블릿_호스트앱_개발자_전달자료_2026-10-19.md

---

1. 결과 요약

- **최종 시험 결과 (2026-10-19 17:43 시작, 20:02 시점까지 약 2시간 20분)**: 펌웨어 v1.1(IWDG, 틱 레이스 수정, 버전 LED 반영) 적용 상태에서 호스트 연결 끊김이 한 번도 없었다. 수정 전에는 약 1.3시간에 8회 끊겼다.
- 인터폰 라인을 프론트엔드 MCU가 ADC로 직접 샘플링하고, 메인 MCU가 상태를 판정하여 USB로 보내는 구조를 완성했다.
- 2026-10-19 실기판 시험에서 새 USB 케이블로 연결하여 Windows 호스트 프로그램과의 통신이 정상 동작함을 확인했다. (이전의 통신 불량은 아래 5절과 같이 연결 대상과 케이블 문제였고, 코드 결함이 아니었다.)
- 호스트가 받는 값은 **MCU가 판정한 상태 + Tx/Rx(Power 대비 0.1%) + Power 전압(0.1V)** 이며, 일반적인 경우 MCU 판정을 그대로 쓰고, 조건이 다른 현장에서는 같은 값으로 호스트가 재판정할 수 있다.

---

2. 시스템 구성

```
인터폰 라인(Tx/Rx/Power)
   |  분압 1/7.98, 고임피던스 분기
   v
프론트엔드 MCU  STM32G030C6T6   (ADC 3채널, 50ms 주기, UART 38400bps)
   |  절연 배리어 ISO7740FDB (UART 1채널 사용, 나머지 예비)
   v
메인 MCU  STM32F103C8T6  (프로젝트명 INTAB_FW: 수신, 상태판정, USB 응답)
   |  USB-CDC (가상 COM)
   v
호스트: Windows(Python) / Android 태블릿
```

구분	내용
절연	필드측(프론트엔드)과 로직측(메인)은 갈바닉 절연. 프론트엔드 전원은 절연 DC-DC + 3.3V LDO
메인 MCU 전원	USB 버스전원과 독립 SMPS 이중화 (쇼트키 OR-ing)
클럭(메인)	HSE 8MHz x PLL(x6) = 48MHz, USB 클럭 48MHz
워치독(메인)	IWDG 약 0.82초, 메인 루프에서 갱신

2.1 메인 MCU 핀 배정 (INTAB_FW 기준)

기능	핀	비고
프론트엔드 UART 수신	PB7 (USART1_RX 리맵)	38400bps, Receive Only, PB6은 미사용
USB D- / D+	PA11 / PA12	
USB 연결 제어	PA8	High = 끊기, Low = 연결 (D+ 풀업 개폐)
USB 5V 감지	PA9	High = 케이블 연결됨
상태 LED	PA3	

---

3. 통신 프로토콜 요약

3.1 호스트 <-> 메인 MCU (USB-CDC, 요청-응답)

방향	프레임
요청 (4B)	`0D 52 0D 0A` (STX, 'R', CR, LF). 정의되지 않은 CMD는 무응답

응답 (16B, 전부 ASCII, SUM 없음)	`0D` + 상태(1) + Tx(4) + Rx(4) + Power(4) + `0D 0A`

필드	의미
상태	`'0'` 대기 / `'1'` 호출 / `'2'` 통화 / `'3'` 전원 이상 / `'4'` 프론트엔드 통신 이상
Tx, Rx	채널 ADC / Power ADC, 0.1% 단위, `"0000"`~`"1000"`
Power	라인 전압 x 10 (0.1V 단위), `"0000"`~`"0263"`

- 전원 이상(`'3'`)에서는 Tx/Rx = `"0000"`, Power는 실측값. 통신 이상(`'4'`)에서는 전 필드 `"0000"`.
- 예시(대기, Tx 3.2%, Rx 93.1%, 13.2V): `0D 30 30 30 33 32 30 39 33 31 30 31 33 32 0D 0A`
- **호스트는 100ms 이하 주기로 폴링한다.** 부팅 직후와 상태 변화 시 MCU가 자발적으로 보내지 않는다.
- **무응답 10초 이상이면 MCU가 USB를 3초간 끊었다 다시 연결하며, 호스트가 계속 요청하지 않으면 반복한다.** (USB 케이블이 없는 경우는 수행하지 않음)

3.2 프론트엔드 -> 메인 MCU (UART, 자율 스트리밍)

10바이트, 50ms 주기: `0xFF` + (Tx 상/하) + (Rx 상/하) + (Power 상/하) + XOR(데이터 6바이트) + `0D 0A`. 메인 MCU는 바이트 단위 상태머신으로 파싱하고, 오버런 등 UART 오류 시 수신을 자동 재개한다. 유효 프레임이 500ms 이상 없으면 상태 `'4'`.

---

4. 상태 판정 알고리즘 (MCU 기본, 호스트 재판정 가능)

입력은 Power ADC 대비 비율(0.1%)이라 12V/24V 겸용이다. 판정은 프론트엔드 프레임(50ms)마다 수행한다.

기준	값
TxH	Tx > 75.0% (TxL은 그 외)
RxL	Rx < 38.0%
RxH	Rx > 75.0%
지정시간	400ms 연속 (프레임 tick 기준)
전원 이상	Power ADC 원시값 < 200 (약 1.3V 미만), 최우선

현재	다음	조건	지정시간
대기	통화	TxH 그리고 Tx > Rx	400ms
대기	호출	TxL 그리고 RxL	400ms
호출	통화	TxH 그리고 Tx > Rx	400ms
호출	대기	Rx > Tx 그리고 RxH	즉시
통화	대기	Rx > Tx 그리고 TxL (RxL이면 제외)	즉시
통화	호출	TxL 그리고 RxL	400ms

- 어느 조건도 아니면 현재 상태를 유지한다. RxL(호출)이 "Rx > Tx(대기)"보다 우선한다.
- 전원 이상, 통신 이상에서 복구되면 대기에서 다시 시작한다.
- 통화 종료 직후 Rx가 복구되는 동안의 순간적인 TxL+RxL은 통화->호출 천이의 400ms 지정시간으로 걸러진다.

---

5. 개발 과정에서 확인한 사항

구분	내용	조치 / 결과
프로토콜	23바이트 16진 ADC 방식에서 16바이트 ASCII(상태 + Tx/Rx 비율 + Power)로 변경	사양서 v2.1, 프로토콜 v1.5 반영
UART 수신	오버런이 나면 HAL이 수신을 중단하고 스스로 재개하지 않음	`HAL_UART_ErrorCallback`에서 `FrontendLink_OnError()` 호출로 재개, PC 폴더 적용
호스트 판정	MCU와 호스트 판정 결과 비교 시 차이 발생	원인은 시험 방식(샘플링 간격, 초기화 누락). 50ms 프레임 150개 입력 시 불일치 0건
소스 줄바꿈	`stm32f1xx_it.c`가 CRLF라 git diff가 전체 변경으로 보임	실제 변경은 5줄. 내용에는 문제 없음
통신 불량(2026-10-17)	호스트가 0바이트 응답	선택한 COM6가 개발 기판이 아닌 다른 USB-시리얼 장치였음
USB 미인식	기판이 USB 장치로 잡히지 않음	USB 케이블 데이터선 불량(충전만 되는 케이블). 새 케이블로 교체 후 정상

5.1 안정화 작업 (2026-10-19)

(1) 독립 워치독(IWDG) 적용

구분	메인 MCU (F103)	프론트엔드 MCU (G030)
타임아웃	약 0.82초 (분주 8, 리로드 4095)	약 0.51초 (분주 4, 리로드 4095)

시작 시점	`main()` 초반 `Wdg_EarlyStart()`	동일
갱신 위치	메인 루프 `AppMonitor_Poll()`	`AppFrontend_Poll()`
추가 감시	USB 소프트 디스커넥트 중에도 갱신	UART 전송완료(TxC)가 300ms 이상 없으면 `NVIC_SystemReset()`
디버그	디버그 정지 시 IWDG 동결	동일 (G0은 DBGMCU 클럭 enable 필요)
리셋 원인	`g_reset_cause` 변수에 보관	동일

(2) 간헐적 USB 끊김 원인 수정

- **증상**: 호스트와의 USB 연결이 간헐적으로 끊김 (약 1.3시간에 8회).
- **원인**: 인터럽트에서 갱신되는 `last` 시각을 `HAL_GetTick() - last`로 직접 빼는 부분에서, 읽는 도중 값이 갱신되면 부호 없는 정수 래핑으로 결과가 약 4.29e9가 되어 "10초 무요청"으로 오판했고, 소프트 디스커넥트(3초)가 실행되었다.
- **수정**: `last` 값을 먼저 로컬 변수에 복사한 뒤 틱과 비교한다. 메인(`AppMonitor_Poll`)과 프론트엔드(`AppFrontend_Poll`) 양쪽에 적용했다.
- **검증**: 수정 후 연속 시험에서 끊김 0회.

(3) 부팅 직후 펌웨어 버전 LED 표시 (버전 1.1)

단계	내용
1	2초 OFF
2	Major 번호 표시
3	1초 OFF
4	Minor 번호 표시
5	1초 OFF
6	평상시(RUN) 표시로 전환

- 번호 표시는 0.1초 ON + 0.3초 OFF를 1세트로 하여 번호 수만큼 반복한다. 마지막 세트 뒤 0.3초 OFF에 1초 OFF가 이어져 번호 뒤 OFF는 총 1.3초이며, 버전 1.1의 표시 시간은 4.8초이다. 번호가 0이면 깜빡임이 없다. 소수부는 정수로 표기한다(1.10이면 10회).
- 버전은 `fw_version.h`의 `FW_VERSION_MAJOR`, `FW_VERSION_MINOR` 두 값으로 정의한다 (현재 1, 1). 두 MCU가 같은 규칙과 같은 모듈(`app_ver_led.c/h`)을 쓴다.
- 부팅 중에는 USB 끊김 표시보다 버전 표시가 우선한다.
- 시험 중 버전 표시가 다시 나타나면 MCU 리셋이 있었다는 뜻이다. 이때 `g_reset_cause`로 원인을 구분한다.

---

6. 검증 현황

항목	방법	결과
상태머신, 프레임 생성	호스트 PC에서 C 시뮬레이션 (`fwtest`)	전 항목 통과
UART 오버런/프레이밍 복구	시뮬레이션 (`uarttest`)	전 항목 통과
호스트 프로토콜 파서, 판정	`test_protocol.py`	통과
MCU 판정 vs 호스트 판정	150프레임 교차 비교	불일치 0
실기판 + Windows 호스트	2026-10-19 현장 시험	정상 동작 (사용자 확인)
버전 LED 시퀀서	호스트 PC 시뮬레이션 (1ms, 50ms 틱; 버전 1.1, 0.0, 2.3, 1.10, 0.2)	1.1은 4.8초, 두 틱 주기의 타이밍 일치
연속 안정성	실기판 + Windows 호스트, 17:43 시작	20:02까지 끊김 0회 (사용자 확인)
Android 앱	빌드 및 실기 시험	**미수행** (코드 작성만, 컴파일 검증 못함)

---

7. 호스트 프로그램

7.1 Windows 모니터 (`host/windows_monitor`)

- Python 3 + pyserial + Tkinter. `python app.py`로 실행, 포트 선택 후 연결.
- 100ms 폴링, Tx/Rx 막대, Power, MCU 판정과 호스트 판정 비교, 임계값 변경 가능한 호스트 판정 모드.
- 연속 5회 응답 실패 시 포트를 자동 재오픈한다.
- 포트 선택 시 우리 기판은 VID:PID `0483:5740`이다. 확인: `python -m serial.tools.list_ports -v`

7.2 Android (`IntercomMonitor`)

- 테스트용 참고 소스(Kotlin, USB Host API 직접 사용). 별도 엔지니어가 정식 앱을 개발할 예정이며, 전달자료는 별도 문서에 정리했다.

---

8. 미결 사항 / 향후 과제

번호	내용	비고
1	양산용 USB VID/PID 확정	현재 ST 데모값(0x0483/0x5740). 변경 시 호스트 앱의 필터도 함께 수정
2	24V 현장 동작	실측 못함. 현재는 Power 대비 75%/38% 유지. 투입 시 재확인하고, 호스트 재판정으로 먼저 시험 가능
3	전원 이상 / 통신 이상 LED 패턴	두 상태 공용 임시값(0.15초 ON/OFF). 구분 여부 확정 필요
4	Android 앱 컴파일 및 실기 시험	전달 후 엔지니어가 진행
5	프론트엔드 MCU 디버그 모드에서 HSE 기동 실패 현상	원인 미규명 (정상 실행에서는 영향 없음)
6	소스 주석 정리	`app_frontend_link.h` 주석의 "PA9/PA10"은 과거 표기. 실제는 PB6/PB7 리맵
7	git 커밋	요청이 없어 수행하지 않음. 현재 PC의 수정 소스는 미커밋 상태
8	프론트엔드 LED(PB0) ON 레벨	버전 표시에서 High=ON으로 가정(`LED_ON_LEVEL`). 반대로 보이면 `GPIO_PIN_RESET`으로 변경
9	선택 개선 (미적용)	프론트엔드 TIM3 기반 시간 기준, 메인 1ms 틱 기반 워치독 갱신

---

9. 산출물 목록

구분	파일 / 폴더
메인 MCU 펌웨어	`app_frontend_link`, `app_state_machine`, `app_monitor`, `app_ver_led` (.c/.h), `fw_version.h`, 통합 가이드
프론트엔드 펌웨어	`app_frontend`, `app_ver_led` (.c/.h), `fw_version.h`, 프론트엔드_MCU_개발사양서.md
사양서	인터폰_상태감지_개발사양서.md (v2.2), 통신_프로토콜_정리.md (v1.5)
Windows 호스트	`windows_monitor` (app.py, protocol.py, test_protocol.py, README)
Android 참고 소스	`IntercomMonitor` (Protocol.kt, MainActivity.kt 등)
시험 코드	`fwtest`, `uarttest` (시뮬레이션)
전달 자료	태블릿_호스트앱_개발자_전달자료_2026-10-19.md / .pdf